

Reviewer Pack — Edge-Expansion Defect Predicts Resolution Width of Tseitin o...

Entry c3c56f86d197 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 04:57:26 UTC

Edge-Expansion Defect Predicts Resolution Width of Tseitin on Random 3-Regular Graphs

Entry ID: c3c56f86d197 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-27 04:57:26 UTC

1. Conjecture

Field A (mathematical branch): Discrete differential geometry via vertex-weighted Cheeger defect: the gap between the bottleneck edge-isoperimetric ratio $h(G)$ and its second-eigenvalue Cheeger upper bound $2\sqrt{d\lambda_2(L)}$ on the unnormalized graph Laplacian **Field B** (complexity object): Resolution refutation width of Tseitin formulas $T(G,c)$ on small 3-regular graphs with odd total charge

Statement:

Let G be a connected 3-regular graph on n vertices with odd-charge assignment c , let $h(G) = \min_{|S| \leq n/2} |E(S, V \setminus S)|/|S|$ be its edge-isoperimetric number, and let $\beta(G) = 2\sqrt{3\lambda_2(L_G)}$ be the Alon-Milman/Cheeger upper bound. Define the Cheeger defect $\Delta(G) = \beta(G) - h(G) \geq 0$. We conjecture that the minimum resolution refutation width $w(T(G,c))$ satisfies $(1/3)h(G)(n - \Delta(G)n/\beta(G)) - 2 \leq w(T(G,c)) \leq h(G)n/2 + 2$ for every odd-charge c , and in particular the *defect-corrected* lower bound $h(G)(1 - \Delta(G)/\beta(G))n/3 - 2$ is tight up to constants on random 3-regular graphs with high probability.

Rationale (proposer's reasoning):

Ben-Sasson-Wigderson tied resolution width to expansion of the underlying constraint hypergraph, but the standard bound uses $h(G)$ only and is loose when the spectral gap fails to certify expansion globally; the Cheeger defect $\Delta(G)$ quantifies exactly how much the linear-algebraic certificate underestimates the combinatorial bottleneck, which we hypothesize is the slack absorbed by short resolution refutations exploiting near-bisecting cuts. Because both $h(G)$ and $\lambda_2(L_G)$ are computable for $n \leq 20$ by brute-force enumeration over subsets and a 20×20 eigensolver, the bridge is fully constructive. The bridge is genuinely new because existing Tseitin lower bounds use $h(G)$ directly or pseudo-expectation arguments and do not isolate the spectral-vs-isoperimetric gap as a width predictor.

Taxonomy category: PROOF_COMPLEXITY_TSEITIN (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 944350bca6fbf921

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 7 sizes $n \in \{8, 10, 12, 14, 16, 18, 20\} \times 30$ random 3-regular graphs (210 instances), each with random odd charge c : SUPPORTED iff (a) $\geq 80\%$ of instances satisfy $\max(0, L(G)) \leq w(T(G, c)) \leq h(G) \cdot n/2 + 2$, AND (b) Pearson $r(L(G), w) > 0.6$. FALSIFIED iff any instance has $w(T(G, c)) < L(G) - 1$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.92	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Tseitin formula resolution width expander 3-regular graph - edge isoperimetric Cheeger inequality resolution refutation lower bound - Tseitin random regular graph proof complexity spectral expansion

Top relevant hits considered: - [<http://arxiv.org/abs/2103.09609v1>] Characterizing Tseitin-formulas with short regular resolution refutations - [<http://arxiv.org/abs/0908.1772v5>] Some Probabilistic Results on Width Measures of Graphs - [<http://arxiv.org/abs/astro-ph/0407622v3>] A Study of the Composition of Ultra High Energy Cosmic Rays Using the High Resolution Fly's Eye - [<http://arxiv.org/abs/1712.06904v3>] Rigidity for the isoperimetric inequality of negative effective dimension on weighted Riemannian manifolds - [<http://arxiv.org/abs/2111.02944v1>] A sharp isoperimetric inequality in metric measure spaces with non-negative Ricci curvature - [<http://arxiv.org/abs/2312.13058v4>] On the Cheeger inequality in Carnot-Carathéodory spaces - [<http://arxiv.org/abs/2501.18019v2>] An exact closed walks series formula for the complexity of regular graphs and some related bounds - [<http://arxiv.org/abs/2301.10131v2>] Random perfect matchings in regular graphs

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
import itertools

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_3_regular_graph(n):
        if n % 2 != 0 or n < 4:
```

```

        raise ValueError("n must be even and >= 4")
graph = [[] for _ in range(n)]
edges = set()
while len(edges) < n * 3 // 2:
    u, v = random.sample(range(n), 2)
    if (u, v) not in edges and (v, u) not in edges and u != v:
        graph[u].append(v)
        graph[v].append(u)
        edges.add((u, v))
return graph

def compute_h(G):
    n = len(G)
    min_cut = float('inf')
    for S in itertools.combinations(range(n), n // 2 + 1):
        cut_size = sum(len([v for v in G[u] if u in S and v not in S]) for u in S)
        min_cut = min(min_cut, cut_size / len(S))
    return min_cut

def power_iteration(A, k=20):
    n = len(A)
    x = [random.random() for _ in range(n)]
    x /= math.sqrt(sum(x[i] ** 2 for i in range(n)))
    for _ in range(k):
        y = [sum(A[i][j] * x[j] for j in range(n)) for i in range(n)]
        y /= math.sqrt(sum(y[i] ** 2 for i in range(n)))
        x, y = y, x
    return x

def compute_lambda_2(G):
    n = len(G)
    A = [[0 if u != v else -len(G[u]) + sum(1 for w in G[v] if w != u) for v in range(n)] for u in range(n)]
    eigenvector = power_iteration(A)
    lambda_2 = max(sum(A[i][j] * eigenvector[j] for j in range(n)) * eigenvector[i] for i in range(n))
    return lambda_2

def compute_delta(G):
    beta_G = 2 * math.sqrt(3 * compute_lambda_2(G))
    h_G = compute_h(G)
    return max(beta_G - h_G, 0)

def compute_L(G):
    n = len(G)
    delta_G = compute_delta(G)
    beta_G = 2 * math.sqrt(3 * compute_lambda_2(G))
    h_G = compute_h(G)
    return h_G * (1 - delta_G / beta_G) * n // 3 - 2

def is_valid_clause(clause, assignment):
    return any(assignment[var] == val for var, val in clause)

def resolve_clause(clause, assignment):
    for i in range(len(clause)):
        if clause[i][1] != assignment[clause[i][0]]:
            new_clause = [c for j, c in enumerate(clause) if j != i]
            return new_clause

```

```

    return []

def dpll(G, n, k, assignment):
    if not any(is_valid_clause(clause, assignment) for clause in G):
        return True
    var = next(v for v in range(n) if v not in assignment)
    for val in [0, 1]:
        new_assignment = assignment.copy()
        new_assignment[var] = val
        if dpll(G, n, k, new_assignment):
            return True
    return False

def compute_w(G, n, k):
    for w in range(k + 1):
        if all(dpll(G, n, w, {}) for _ in range(30)):
            return w
    return -1

n = random.choice([8, 10, 12, 14, 16, 18, 20])
G = generate_random_3_regular_graph(n)
c = {i: random.randint(0, 1) for i in range(n)}

h_G = compute_h(G)
lambda_2_G = compute_lambda_2(G)
delta_G = compute_delta(G)
L_G = compute_L(G)

w_T_G_c = compute_w(G, n, n // 2)

if w_T_G_c < max(0, L_G) - 1:
    return {
        "metric_name": "w(T(G,c))",
        "metric_value": w_T_G_c,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": f"Instance {n} with h(G)={h_G}, lambda_2(G)={lambda_2_G}, delta(G)={delta_G}"
    }

return {
    "metric_name": "w(T(G,c))",
    "metric_value": w_T_G_c,
    "instances_tested": 1,
    "conjecture_holds": True,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [11, 23, 37, 53, 71]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

```

```

metric_values = [r["metric_value"] for r in results if "metric_value" in r]
conjecture_holds = all(r["conjecture_holds"] for r in results if "conjecture_holds" in r)

mean = sum(metric_values) / len(metric_values)
std = math.sqrt(sum((x - mean) ** 2 for x in metric_values) / len(metric_values))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if conjecture_holds or support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
elif any(r["w(T(G,c))"] < max(0, r["L(G)"]) - 1 for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if result["w(T(G,c))"] < max(0, r["L(G)"]) - 1)
    print(f"RESULT: FALSIFIED counterexample=\"Instance with w(T(G,c)) < L(G) - 1\" first_fail={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_59e871f1.py", line 131, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_59e871f1.py", line 102, in run_trial
    lambda_2_G = compute_lambda_2(G)
                 ^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_59e871f1.py", line 54, in compute_lambda_2
    eigenvector = power_iteration(A)
                  ^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_59e871f1.py", line 44, in power_iteration
    x /= math.sqrt(sum(x[i] ** 2 for i in range(n)))

```

TypeError: unsupported operand type(s) for /=: 'list' and 'float'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` in `power_iteration` (dividing a Python list by a float) before any instances were evaluated, so neither the support nor falsification criteria could be assessed. | next: Fix `power_iteration` to use a numpy array (or convert `x` to an array before normalization) and rerun the 210-instance sweep.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	24687
2	preregistration	claude_max	opus	0	0	6131
3	novelty	claude_max	opus	0	0	3144
4	novelty	claude_max	opus	0	0	9189
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17052
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16703
7	judge	claude_max	opus	0	0	4694

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 81600 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in research/audit/c3c56f86d197.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/c3c56f86d197.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/c3c56f86d197.tar.gz (if generated)